DEVELOP *like a girl*

ADVANCED KIT / LEARN

Getting started

A practical guide to your first session.

01 Install → 02 Plan → 03 Build

01

Install and verify your kit

By the end, Core and your selected builder will be available in a Claude Code session opened in your own project folder.

Before you begin

Use the official Claude Code setup guide if you have not installed and signed into Claude Code. Have Git available for the GitHub marketplace. You can check both from your terminal:

```
claude --version
git --version
```

Both should print a version. If either command is missing, complete that tool's setup before continuing. The examples below use the terminal installation route. Claude Code also has a plugin panel; its current options are documented in Install and manage plugins.

Access: This marketplace is private. Your GitHub account must have access, and Git on this machine must be authenticated for that account. Advanced is self-contained and does not require Starter.

1. Add the marketplace and install Core

Run in your terminal:

```
claude plugin marketplace add https://github.com/legioncodeinc/develop-like-a-girl-advanced-resources.git
claude plugin install dlag-advanced-core@dlag-advanced
```

Adding a marketplace makes its plugins discoverable. Installing Core loads the shared foundation. The default user installation makes it available to you across projects; use the plugin panel if you need a different installation scope.

2. Install your builder

For public pages, service websites, content, or inquiry forms:

```
claude plugin install dlag-advanced-website-builder@dlag-advanced
```

For signed-in workflows, data screens, onboarding, search, or integrations:

```
claude plugin install dlag-advanced-app-builder@dlag-advanced
```

You can install both. Keep Core installed and enabled alongside either builder.

3. Open your own project

Open a terminal in the folder containing your project. For a new project, create and open an empty folder using your usual file manager. Then launch:

```
claude
```

You do not need to clone this marketplace to use it. Avoid building inside its `plugins/` directory. This repository distributes the kit; your project has its own files.

4. Confirm availability

Inside Claude Code, enter:

```
/plugin
```

Confirm `dlag-advanced-core` and your chosen builder appear as installed and enabled. Then send:

```
/dlag-advanced-core:get-started Inspect this folder and propose a first task. Keep the
proposal in chat.
```

A useful result describes the current folder and recommends a next task. It is a setup proposal, not a newly built app. If the command is unavailable, start a fresh session in the project folder and check the plugin panel again. Keep the full `dlag-advanced-core:` prefix.

If installation stops

A repository-not-found or permission error can mean your GitHub account lacks access or Git is authenticated as another account. Confirm access to this private repository with its owner before retrying. Do not paste access tokens into a prompt or issue. Troubleshooting has the next checks.

Next: Your first session.

Your first session

By the end of this walkthrough, you will have a small project brief and a first task you can approve. Use the example as practice, then replace it with your own idea.

Before you start: Install Core and open Claude Code in your project folder. This exercise can start in an empty folder and does not need a provider account.

1. Describe one person and one job

Send:

```
/dlag-advanced-core:get-started I want a website for a neighborhood pottery class. A
visitor should understand the class and request a place. Inspect this folder, preserve
existing work, and recommend one first task. Keep the proposal in chat.
```

Look for a short account of the folder, any missing information, and a recommendation. It is reasonable for the studio to ask about the class details, audience, or visual direction. Answer the questions that change the first task.

2. Turn the idea into a first version

Send:

```
/dlag-advanced-core:plan-project Plan the first version of the pottery-class website.
Start with one public page showing the class, date, price, and contact details. Use
clearly marked draft copy for missing facts. Save the draft to docs/first-version.md in
this project. If that file already exists, show me how you would update it first. Defer
booking, payment, and email automation.
```

The requested Markdown file should capture the outcome, essential content, deferred ideas, and checks. If the session cannot write files, it should provide the complete draft and say it is unsaved. Confirm the actual saved path before moving on.

3. Make the task testable

Ask:

```
Refine the first task so I can tell when it is finished. Include the mobile layout,
keyboard navigation, clear contact action, and what should happen when content is
missing. Keep the first task limited to the public page.
```

“Build the site” is too broad to review. “A visitor can read the class information on a phone and find the contact action” is something you can check. A plan is still a draft until you accept it.

4. Choose the implementation step

Read the draft, correct the facts, and accept the first task when it matches your intent. Then follow Build a website. That guide uses the Website Builder to author a local slice. Project setup and planning do not scaffold or deploy the application for you.

Before you leave the session

Ask for the saved paths, accepted decisions, open questions, and one next task. Keep this short enough that you can use it to restart tomorrow. If the discussion grows into accounts, checkout, and automation, move those ideas to the deferred list and finish the public page first.

Next: Build a website, or Build an app for a signed-in product.

Continue with the learning hub for website, app, notebook, and release guides. The Markdown guides are the editable source for this edition of the PDF.